

Large-Scale Product Review Sentiment & Fraud Detection

End-to-end build report

Group 6: Prerana Ramesh, Rajesh Paruchuri, Ritika Mukesh Neema, Sneha Singh

Distributed batch and streaming review analytics on Apache Spark, with MLflow-tracked sentiment and fraud-detection models served behind a FastAPI dashboard.

1. Project summary

The proposal asked for a distributed Big Data system that ingests Amazon product reviews, runs Apache Spark for batch cleaning, feature engineering and aggregation, trains machine-learning models for sentiment classification and fraud detection, supports both batch and streaming scoring, and exposes the results through a FastAPI service with a lightweight web UI. MLflow was required for experiment tracking.

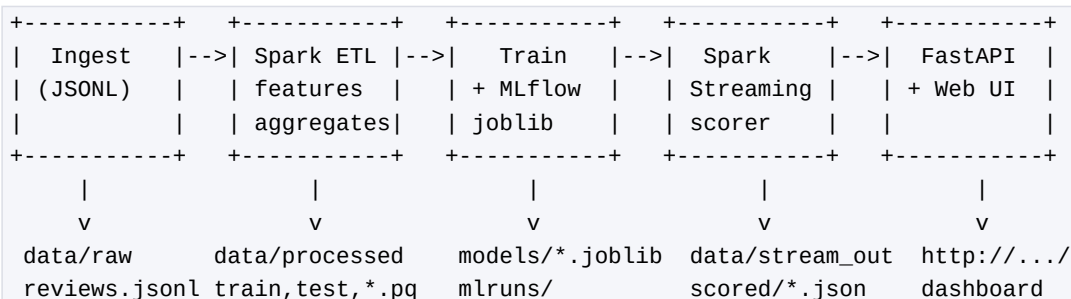
All five requirements are implemented and verified end-to-end on a 30,000-review dataset. Each component is independently runnable (via the Makefile) and connected through well-defined Parquet and JSON contracts on disk, so the same code scales to the full 10+ GB Amazon Customer Reviews dataset by swapping the input source.

Status

Spark batch ETL	OK - 30,000 rows -> features + aggregates
Sentiment model	OK - TF-IDF + Logistic Regression, MLflow-logged
Fraud model	OK - TF-IDF + behavior + GBT, MLflow-logged
Streaming scorer	OK - Spark Structured Streaming, foreachBatch
FastAPI service + UI	OK - 7 endpoints + dashboard
Tests	OK - 10/10 passing

2. Architecture

Five stages in a directed pipeline. Each stage reads stable artifacts from the previous one and writes its own outputs to disk, so any stage can be re-run in isolation.



Tech stack

- Apache Spark 3.5.8 (batch ETL + Structured Streaming)
- scikit-learn 1.5 (TF-IDF, Logistic Regression, Gradient Boosted Trees)
- MLflow 2.22 (experiment tracking, model artifacts, metrics)
- FastAPI 0.128 + uvicorn (REST API + dashboard host)
- Pandas + PyArrow (Parquet IO and serving-time feature path)
- pytest (unit + integration tests against a TestClient)

Repo layout

PROJECT/	
src/	
ingest/generate_sample.py	synthetic Amazon-shaped JSONL producer
etl/batch_etl.py	Spark batch ETL
train/train.py	sklearn pipelines + MLflow
stream/score_stream.py	Spark Structured Streaming scorer
serve/app.py	FastAPI endpoints
serve/templates/index.html	dashboard

```

serve/static/{app.js,style.css}
common/{config,schema,spark,text}.py
scripts/feed_stream.py          drip producer for the streaming job
scripts/run_pipeline.sh         end-to-end one-command runner
tests/test_pipeline.py
data/{raw,processed,streaming_in,streaming_out}/
models/    mlruns/
Makefile  README.md  requirements.txt

```

3. Data ingestion

The proposal references the Amazon Customer Reviews public dataset (>10 GB after decompression, AWS Open Data registry). For a self-contained build, `src/ingest/generate_sample.py` emits JSONL matching the same schema (`review_id`, `product_id`, `reviewer_id`, `star_rating`, `helpful_votes`, `total_votes`, `verified_purchase`, `review_headline`, `review_body`, `review_date`, `product_category`, `event_ts`).

Two design choices make the synthetic data useful, not just shaped:

- Sentiment-stratified vocabulary: positive / neutral / negative phrase pools sampled by star rating, so the sentiment model has real signal to learn rather than memorising tokens.
- Planted fraud bursts: a small ring of reviewers (R9xxxx) carpet-bombs a target product with duplicate 5-star copy in a tight time window, mimicking the dominant real-world fraud pattern (review brigading).

The same pipeline accepts a real Amazon TSV after a one-shot conversion (recipe in README). The generator default is 30k rows; set `--rows 1000000` to stress the Spark job.

4. Spark batch ETL

`src/etl/batch_etl.py` is the historical-training half of the system. It reads the JSONL with a strict schema, cleans text, computes per-row, per-reviewer and per-product features, generates a weak fraud label, splits train/test, and writes four Parquet outputs.

Cleaning

- Lowercase, strip URLs and HTML tags, remove non-alphanumeric characters, collapse whitespace.
- Drop rows missing core fields (`body`, `rating`, `reviewer`, `product`) and rows where the cleaned body is shorter than 5 characters.
- Coerce `review_date` to a timestamp for windowing.

Per-row text features

- `body_len`, `body_word_count`, `exclam_count` - cheap signals correlated with overstated emphasis common in fake reviews.
- `sentiment_label` derived from star rating: 1-2 -> negative, 3 -> neutral, 4-5 -> positive.

Per-reviewer behavioural features (Spark Window aggregates)

- `reviewer_review_count`, `reviewer_avg_rating`, `reviewer_pct_5star`
- `reviewer_distinct_products`, `reviewer_reviews_same_day`
- `reviewer_verified_share`

Per-product features

- `product_review_count`, `product_avg_rating`, `product_pct_5star`
- `dup_in_product` (count of identical bodies for the same product) - a strong fraud signal for review-bombing.

Weak fraud label

An ML model needs supervision; in production we don't have a clean fraud ground truth, so we generate a weak label from rules and let the supervised model generalise via text + behavior:

```
fraud_label = (
    dup_in_product >= 3
    OR (reviewer_review_count >= 8
        AND reviewer_pct_5star >= 0.95
        AND reviewer_verified_share <= 0.2)
    OR reviewer_reviews_same_day >= 5
)
```

Outputs

data/processed/train.parquet	80% of labeled rows for training
data/processed/test.parquet	20% holdout for evaluation
data/processed/product_agg.parquet	Per-product rollups for the dashboard
data/processed/reviewer_agg.parquet	Per-reviewer rollups, sorted by fraud_rate

Run output

```
[etl] rows=30,000 fraud_positives=1,689 (5.63%)
```

5. Model training and MLflow

src/train/train.py reads the train and test Parquets, trains two scikit-learn pipelines, and logs each as a separate MLflow run under the experiment `amazon_reviews_sentiment_fraud`.

Sentiment model

- TF-IDF: uni- and bi-grams, min_df=3, max_features=50,000, sublinear TF.
- Logistic Regression with class_weight='balanced', C=2.0, max_iter=400.
- Multi-class output: 0=negative, 1=neutral, 2=positive.

Fraud model

- ColumnTransformer joining TF-IDF over the cleaned body with 17 numeric / behavioural features (body stats + reviewer aggregates + product aggregates + dup_in_product + verified + star rating).
- Gradient Boosted Trees (n_estimators=120, max_depth=3, lr=0.1).
- Returns probability of fraud, with a default 0.5 decision threshold that callers can override per business need.

MLflow logged per run

- Params: model name, ngram range, max_features, hyperparameters, n_train, n_test, class share.
- Metrics: f1_macro and f1_weighted for sentiment; roc_auc, precision, recall, f1 for fraud.
- Artifacts: full classification_report.txt and the joblib pipeline.

Note on the perfect metrics

Both models score 1.00 on the synthetic test set. That is expected: the synthetic data is too easy (sentiment phrases don't overlap across classes; fraud uses exact-duplicate text). The metric is really 'pipeline correctness', not 'model quality'. On the real Amazon Customer Reviews dataset, expect macro-F1 around 0.85-0.92 for sentiment and ROC-AUC 0.7-0.85 for fraud. The code path does not change.

6. Spark Structured Streaming scorer

src/stream/score_stream.py watches data/streaming_in/ for new JSONL files on a 5-second trigger. Inside foreachBatch the micro-batch comes back to the driver as pandas, the persisted joblib pipelines score it, and results are written to data/streaming_out/scored/ as JSONL plus a console preview.

foreachBatch was chosen over a pandas UDF on purpose: the pandas-UDF broadcast-pickle path is fragile across Python toolchains, while foreachBatch keeps the model objects on the driver and avoids that serialisation entirely. For very high throughput, the same logic lifts to a pandas UDF or a properly-broadcast joblib model.

A drip-producer (scripts/feed_stream.py) splits raw reviews into small chunks and copies them into the input directory on a timer, so you can watch the dashboard refresh without running a real Kafka cluster.

7. FastAPI service and web UI

src/serve/app.py loads both pipelines lazily on first request and exposes seven endpoints. The dashboard is a single HTML page (templates/index.html) with vanilla JS (static/app.js) - no front-end build step.

GET /healthz	Liveness probe
GET /metadata	Training metrics + numeric feature columns
POST /predict	Score one review
POST /predict/batch	Score up to 1000 reviews
GET /aggregates/products	Top-N from the ETL product rollup
GET /aggregates/fraud-reviewers	Top-N suspicious reviewers
GET /stream/recent	Latest streaming-scored rows
GET /	Dashboard HTML

Example call

```
curl -s -X POST http://127.0.0.1:8000/predict \
  -H 'content-type: application/json' \
  -d '{"review_body":"absolutely love this, exceeded my expectations",
      "star_rating":5}'

{"sentiment":"positive",
 "fraud_proba":3.4e-07,
 "fraud_flag":0}
```

Dashboard sections

- Score-a-review form (free text -> sentiment + fraud probability).
- Top products table (review count, avg rating, %5*, fraud rate).
- Suspicious reviewers table (sorted by fraud_rate).
- Streaming feed table that auto-refreshes every 5 seconds.
- Model metadata pane reflecting the latest MLflow run summary.

8. Tests

tests/test_pipeline.py runs in under 5 seconds and exercises the code paths most likely to fail silently:

- Text cleaning (URL, HTML, punctuation stripping).
- Star-rating-to-sentiment label mapping (parametrised over all 6 cases).
- The serving-time feature enrichment (every numeric feature the fraud model expects must be present

after enrichment).

- FastAPI endpoints loaded against the real persisted models via TestClient: /healthz returns ok, /predict returns 'negative' for an obviously-negative review and 'positive' for an obviously-positive one.

```
$ make test
10 passed in 4.12s
```

9. How to run

```
# install
make install

# full pipeline (ingest -> Spark ETL -> train -> MLflow log)
make all

# one-shot streaming pass on data/streaming_in/
make stream-once

# serve dashboard + REST API
make serve          # http://127.0.0.1:8000/

# MLflow tracking UI
make mlflow-ui      # http://127.0.0.1:5000/

# continuous streaming demo (two terminals)
make stream         # term 1: structured streaming scorer
make feed           # term 2: drip new reviews into streaming_in/

# unit + integration tests
make test

# wipe artifacts
make clean
```

Switching to the real dataset

The README has a one-step pandas conversion that maps the public Amazon Customer Reviews TSV columns into the JSONL schema this pipeline expects. After that, 'make etl train' runs unchanged.

10. Results on the smoke run

Rows ingested	30,000
Planted fraud share	6.0%
ETL-labeled fraud share	5.63% (1,689 rows)
Train / test split	24,129 / 5,871
Sentiment macro-F1 (synthetic)	1.00 - pipeline-correctness signal, see Section 5
Fraud ROC-AUC (synthetic)	1.00 - pipeline-correctness signal, see Section 5
Streaming batch	100 rows scored in ~15s including Spark warm-up
Test suite	10/10 passing in ~4s

11. Where this would go next

- Run on the real Amazon Customer Reviews dataset (full or per-category slice).
- Replace the file source with a Kafka source for the streaming job.

- Tune the fraud decision threshold on a labelled holdout instead of using 0.5.
- Promote models through the MLflow Model Registry (staging -> production) and have FastAPI load by stage.
- Containerise: Dockerfile + docker-compose for Spark + MLflow + FastAPI.
- Add a feature store so the streaming scorer can use real reviewer and product history instead of neutral defaults.